BitonX AI Engineering Specification
Version 1.0

BitonX Engineering Team

July 19, 2026

Contents

1 Project Definition **8**
- 1.1 Project Name . . . 8
- 1.2 Product Name . . . 8
- 1.3 Project Type . . . 8
- 1.4 Primary Objective . . . 8
- 1.5 Target Industry . . . 8
- 1.6 Supported Input Formats . . . 8
- 1.7 Future Supported Formats . . . 9

2 Global Technology Stack **10**
- 2.1 Programming Languages . . . 10
- 2.2 Backend Frameworks . . . 10
- 2.3 Frontend Frameworks . . . 11
- 2.4 Core AI Libraries . . . 11

3 Artificial Intelligence Stack **12**
- 3.1 Computer Vision . . . 12
- 3.2 Deep Learning . . . 12
- 3.3 Machine Learning . . . 13

4 Project Modules **14**
- 4.1 Main Services . . . 14
- 4.2 Development Principles . . . 14

5 Microservices Specification **16**
- 5.1 Vision Service . . . 16
  - 5.1.1 Purpose . . . 16
  - 5.1.2 Responsibilities . . . 16
  - 5.1.3 Supported Inputs . . . 16
  - 5.1.4 Outputs . . . 16
  - 5.1.5 Libraries . . . 17
  - 5.1.6 Algorithms . . . 17
- 5.2 OCR Service . . . 17
  - 5.2.1 Purpose . . . 17
  - 5.2.2 Responsibilities . . . 17
  - 5.2.3 Outputs . . . 18
  - 5.2.4 Libraries . . . 18
- 5.3 Detection Service . . . 18
  - 5.3.1 Purpose . . . 18
  - 5.3.2 Target Objects . . . 18
  - 5.3.3 Libraries . . . 18
  - 5.3.4 Candidate Models . . . 19
- 5.4 Segmentation Service . . . 19
  - 5.4.1 Purpose . . . 19

5.4.2 Responsibilities . . . 19
5.4.3 Libraries . . . 19
5.5 Geometry Service . . . 19
5.5.1 Purpose . . . 19
5.5.2 Responsibilities . . . 19
5.5.3 Mathematics . . . 20
5.6 Knowledge Graph Service . . . 20
5.6.1 Purpose . . . 20
5.6.2 Node Types . . . 20
5.6.3 Relationships . . . 21
5.6.4 Database . . . 21
5.7 Quantity Engine . . . 21
5.7.1 Purpose . . . 21
5.7.2 Responsibilities . . . 21
5.7.3 Outputs . . . 21
5.8 Cost Engine . . . 22
5.8.1 Purpose . . . 22
5.8.2 Responsibilities . . . 22
5.9 Reporting Service . . . 22
5.9.1 Outputs . . . 22

6 Project Architecture **23**
6.1 Architecture Style . . . 23
6.2 Root Project Structure . . . 23
6.3 Backend Structure . . . 24
6.4 AI Structure . . . 25
6.5 Frontend Structure . . . 26

7 Dataset Design **27**
7.1 Dataset Directory . . . 27
7.2 Supported Annotation Types . . . 27
7.3 Object Classes . . . 28

8 Training Pipeline **29**
8.1 Training Workflow . . . 29
8.2 Image Augmentation . . . 30
8.3 Evaluation Metrics . . . 30
8.4 Model Registry . . . 30

9 Training Data Sources **32**
9.1 Primary Datasets . . . 32
9.2 Dataset Download Manager . . . 32
9.3 CubiCasa5K . . . 32
9.4 ResPlan . . . 33

10 Dataset Management **34**
10.1 Dataset Directory . . . 34
10.2 Dataset Loader . . . 34
10.3 Dataset Validation . . . 35

11 Dataset Preprocessing **36**
11.1 Image Resolution . . . 36
11.2 Supported Annotation Types . . . 36

12 Engineering Understanding Engine **38**
12.1 Expected Output . . . 38
12.2 Output Format . . . 38

13 Engineering Computation Engine **39**
13.1 Purpose . . . 39
13.2 Geometry Engine . . . 39
13.3 Wall Quantity Engine . . . 40
13.4 Floor Quantity Engine . . . 40
13.5 Ceiling Engine . . . 40
13.6 Painting Engine . . . 40
13.7 Concrete Engine . . . 41
13.8 Reinforcement Engine . . . 41
13.9 Material Estimation Engine . . . 41
13.10 Cost Estimation Engine . . . 42

14 Engineering Rules Engine **43**

15 Engineering Reporting Engine **44**

16 AI Orchestration Pipeline **45**
16.1 Objective . . . 45
16.2 Global Pipeline . . . 45
16.3 Stage 1 : File Validation . . . 47
16.4 Stage 2 : Image Enhancement . . . 47
16.5 Stage 3 : Drawing Classification . . . 48
16.6 Stage 4 : OCR Pipeline . . . 48
16.7 Stage 5 : Object Detection . . . 48
16.8 Stage 6 : Segmentation . . . 49
16.9 Stage 7 : Geometry Reconstruction . . . 49
16.10 Stage 8 : Engineering Graph . . . 50
16.11 Stage 9 : Engineering Reasoning . . . 50
16.12 Stage 10 : Quantity Computation . . . 50
16.13 Stage 11 : Engineering Validation . . . 51
16.14 Stage 12 : Report Generation . . . 51

17 Engineering Knowledge Base **52**
17.1 Objective . . . 52
17.2 Knowledge Domains . . . 52
17.3 Building Elements . . . 52

18 Database Architecture **54**
18.1 Primary Database . . . 54
18.2 Graph Database . . . 54
18.3 Object Storage . . . 54

19 Training Dataset Sources **56**
19.1 Dataset Lifecycle . . . 56

20 Core Database Tables **58**

21 Engineering Graph **59**

22 AI Model Registry **60**

23 REST API Specification **61**
23.1 API Principles . . . 61
23.2 Authentication Endpoints . . . 61
23.3 Project Endpoints . . . 61
23.4 Upload Endpoints . . . 61
23.5 AI Endpoints . . . 62
23.6 Reports . . . 62

24 API Error Codes **63**

25 Security Requirements **64**

26 Monitoring **65**

27 AI Training Specification **66**
27.1 Objective . . . 66
27.2 Training Strategy . . . 66
27.3 Drawing Classification Model . . . 66
27.4 Object Detection Model . . . 67
27.5 Segmentation Model . . . 67
27.6 OCR Model . . . 68
27.7 Geometry Reconstruction Model . . . 68
27.8 Engineering Reasoning Model . . . 68

28 Training Infrastructure **69**

29 Data Augmentation **70**

30 Loss Functions **71**

31 Evaluation **72**

32 Implementation Roadmap **73**
32.1 Phase 1 . . . 73
32.2 Phase 2 . . . 73
32.3 Phase 3 . . . 74
32.4 Phase 4 . . . 74
32.5 Phase 5 . . . 75
32.6 Phase 6 . . . 75
32.7 Phase 7 . . . 75
32.8 Phase 8 . . . 76
32.9 Phase 9 . . . 76
32.10Phase 10 . . . 76
32.11Phase 11 . . . 77
32.12Phase 12 . . . 77
32.13Phase 13 . . . 77

33 AI Agent Instructions **78**

34 PostgreSQL Database Schema **79**
34.1 Database Overview . . . 79
34.2 users . . . 79
34.3 roles . . . 79
34.4 permissions . . . 80
34.5 projects . . . 80
34.6 uploaded$_{f}iles$ . . . 80

34.7 ai_jobs . . . 81
34.8 detected_objects . . . 81
34.9 geometry . . . 82
34.10 engineering_graph . . . 82
34.11 material_catalog . . . 82
34.12 quantity_results . . . 83
34.13 boq . . . 83
34.14 reports . . . 83
34.15 audit_logs . . . 84
34.16 Indexes . . . 84
34.17 Constraints . . . 84

35 Neo4j Engineering Graph **85**
35.1 Purpose . . . 85
35.2 Node Types . . . 85
35.3 Relationship Types . . . 85
35.4 Building Hierarchy . . . 86
35.5 Wall Relationships . . . 86
35.6 Room Relationships . . . 87
35.7 Node Properties . . . 87
35.8 Relationship Properties . . . 87
35.9 Graph Construction Pipeline . . . 88
35.10 Cypher Query Examples . . . 88
35.11 Graph Validation Rules . . . 89
35.12 Future Extensions . . . 89

36 Frontend Architecture **90**
36.1 Technology Stack . . . 90
36.2 Application Structure . . . 90
36.3 Main Pages . . . 91
36.4 Dashboard Layout . . . 92
36.5 Sidebar Components . . . 92
36.6 Dashboard Widgets . . . 93
36.7 Drawing Viewer . . . 93
36.8 AI Analysis Panel . . . 94
36.9 BOQ Interface . . . 94
36.10 Report Viewer . . . 94
36.11 State Management . . . 95
36.12 API Services . . . 95
36.13 Routing . . . 96
36.14 Role Based UI . . . 96
36.15 Theme System . . . 96
36.16 Responsive Design . . . 97
36.17 Performance Requirements . . . 97
36.18 Accessibility . . . 97

37 System Architecture and Infrastructure **98**
37.1 Architecture Style . . . 98
37.2 Repository Structure . . . 98
37.3 Backend Structure . . . 99
37.4 Frontend Structure . . . 100
37.5 AI Structure . . . 100
37.6 Infrastructure Components . . . 101
37.7 Application Services . . . 102

37.8 Message Queue . . . 102
37.9 Execution Flow . . . 103
37.10Docker Services . . . 104
37.11Kubernetes . . . 104
37.12Development Environments . . . 104
37.13Continuous Integration . . . 105
37.14Continuous Deployment . . . 105
37.15Logging . . . 105
37.16Monitoring . . . 106
37.17Coding Standards . . . 106
37.18Project Quality Requirements . . . 107

38 Engineering Formula Engine 108
38.1 Objective . . . 108
38.2 Design Principles . . . 108
38.3 Formula Categories . . . 108
38.4 Geometry Engine . . . 109
38.5 Concrete Formula Library . . . 110
38.6 Reinforcement Library . . . 110
38.7 Masonry Library . . . 110
38.8 Finishing Library . . . 111
38.9 Unit Conversion Engine . . . 111
38.10Validation Engine . . . 112
38.11Formula Execution Pipeline . . . 112
38.12Formula Registry . . . 113
38.13Country Rule Packs . . . 113
38.14Explainable Engineering . . . 114
38.15Future Extensions . . . 114

A Technology Stack and Version Matrix 115
A.1 Purpose . . . 115
A.2 Programming Languages . . . 115
A.3 Backend . . . 115
A.4 Artificial Intelligence . . . 115
A.5 Frontend . . . 116
A.6 Databases . . . 116
A.7 DevOps . . . 116
A.8 Development Tools . . . 117
A.9 Official Datasets . . . 117
A.10 Official Development Principles . . . 117

I Engineering Governance and AI Development Contract 118

B Project Scope (Version 1.0) 119
B.1 Mission . . . 119
B.2 Supported Features . . . 119
B.3 Out of Scope . . . 120

C Engineering Rule Library 121
C.1 Rule Categories . . . 121
C.2 Engineering Validation . . . 121
C.3 Explainable Engineering . . . 122

D Country Rule Packs 123

E Product Roadmap **124**

F AI Development Contract **125**
F.1 Architecture 125
F.2 Implementation 125
F.3 Engineering Constraints 126
F.4 Final Objective 126

Chapter 1

Project Definition

1.1 Project Name

BitonX

1.2 Product Name

BitonX Quant

1.3 Project Type

Cloud Native Artificial Intelligence Platform

1.4 Primary Objective

Develop a production-grade web platform capable of understanding engineering drawings, reconstructing construction elements digitally, calculating quantities automatically, estimating project costs, generating Bills of Quantities (BOQ), and providing AI-assisted engineering analysis.

1.5 Target Industry

- Construction Technology
- Civil Engineering
- Quantity Surveying
- Structural Engineering
- Building Information Processing

1.6 Supported Input Formats

- PDF
- PNG
- JPEG

1.7 Future Supported Formats

- DWG
- DXF
- IFC

Chapter 2

Global Technology Stack

2.1 Programming Languages

- Python 3.12+
- TypeScript
- JavaScript
- SQL
- HTML5
- CSS3
- Bash

2.2 Backend Frameworks

- FastAPI
- Pydantic
- SQLAlchemy
- Alembic
- Celery
- Redis
- RabbitMQ
- PostgreSQL
- Docker
- Docker Compose
- Kubernetes
- JWT
- OAuth2
- Nginx

2.3 Frontend Frameworks

- React
- Next.js
- TypeScript
- TailwindCSS
- ShadCN UI
- React Query
- Axios
- Zustand
- React Hook Form
- Zod
- Framer Motion

2.4 Core AI Libraries

- PyTorch
- TensorFlow
- OpenCV
- NumPy
- Pandas
- SciPy
- Matplotlib
- Scikit-learn
- Pillow
- Albumentations
- OpenMMLab
- HuggingFace Transformers
- ONNX Runtime

Chapter 3

Artificial Intelligence Stack

3.1 Computer Vision

- Image Classification
- Object Detection
- Instance Segmentation
- Semantic Segmentation
- OCR
- Line Detection
- Contour Detection
- Perspective Correction
- Image Enhancement
- Drawing Understanding

3.2 Deep Learning

- Convolutional Neural Networks (CNN)
- Vision Transformers (ViT)
- ResNet
- EfficientNet
- ConvNeXt
- YOLO
- RT-DETR
- DETR
- Mask R-CNN
- Segment Anything Model (SAM)
- GroundingDINO
- Florence-2
- DINOv2

3.3 Machine Learning

- Supervised Learning
- Transfer Learning
- Semi-Supervised Learning
- Self-Supervised Learning
- Active Learning
- Continual Learning
- Hyperparameter Optimization
- Ensemble Learning

Chapter 4

Project Modules

4.1 Main Services

1. Authentication Service
2. User Service
3. Project Service
4. Upload Service
5. Storage Service
6. Vision Service
7. OCR Service
8. Detection Service
9. Segmentation Service
10. Geometry Service
11. Quantity Engine
12. Cost Engine
13. Reporting Service
14. AI Assistant
15. Notification Service
16. Billing Service
17. Administration Service

4.2 Development Principles

- AI-First Architecture
- Modular Architecture
- Clean Architecture
- Microservices
- API-First Design

- Scalability
- High Performance
- Security by Design
- Explainable AI
- Production Ready

Chapter 5

Microservices Specification

5.1 Vision Service

5.1.1 Purpose

Responsible for image preprocessing and normalization before any AI inference.

5.1.2 Responsibilities

- Validate uploaded files
- Convert PDF pages into images
- Normalize image resolution
- Remove noise
- Improve contrast
- Correct perspective distortion
- Detect drawing borders
- Remove unnecessary margins
- Rotate drawings automatically
- Detect drawing orientation
- Produce normalized tensors

5.1.3 Supported Inputs

- PDF
- PNG
- JPEG

5.1.4 Outputs

- Clean Image
- Enhanced Image
- Image Metadata
- Resolution

- Rotation Angle
- Scale Candidate

5.1.5 Libraries

- OpenCV
- NumPy
- Pillow
- pdf2image
- SciPy

5.1.6 Algorithms

- Adaptive Thresholding
- Gaussian Blur
- CLAHE
- Edge Detection
- Hough Transform
- Morphological Operations
- Perspective Transform
- Histogram Equalization

5.2 OCR Service

5.2.1 Purpose

Extract every textual element from engineering drawings.

5.2.2 Responsibilities

- Detect text regions
- Detect dimensions
- Extract room names
- Extract notes
- Extract annotations
- Detect symbols
- Detect scale information
- Detect floor labels
- Detect section labels
- Detect drawing title

5.2.3 Outputs

- JSON Text
- Bounding Boxes
- Confidence Score
- OCR Metadata

5.2.4 Libraries

- PaddleOCR
- EasyOCR
- Tesseract
- OpenCV

5.3 Detection Service

5.3.1 Purpose

Detect all engineering objects contained inside construction drawings.

5.3.2 Target Objects

- Walls
- Doors
- Windows
- Columns
- Beams
- Slabs
- Rooms
- Stairs
- Dimensions
- Symbols
- Furniture
- Plumbing Symbols
- Electrical Symbols

5.3.3 Libraries

- Ultralytics
- PyTorch
- OpenCV
- ONNX Runtime

5.3.4 Candidate Models

- YOLO
- RT-DETR
- GroundingDINO
- Faster R-CNN

5.4 Segmentation Service

5.4.1 Purpose

Generate precise object masks.

5.4.2 Responsibilities

- Wall segmentation
- Room segmentation
- Door segmentation
- Window segmentation
- Stair segmentation
- Column segmentation
- Beam segmentation
- Slab segmentation

5.4.3 Libraries

- Detectron2
- SAM2
- OpenMMLab
- PyTorch

5.5 Geometry Service

5.5.1 Purpose

Convert detected pixels into engineering geometry.

5.5.2 Responsibilities

- Compute wall length
- Compute wall thickness
- Compute room area
- Compute slab area
- Compute beam length

- Compute column dimensions
- Compute opening dimensions
- Compute perimeter
- Compute volume
- Build geometric relationships

5.5.3 Mathematics

- Euclidean Geometry
- Computational Geometry
- Matrix Algebra
- Coordinate Systems
- Graph Theory

5.6 Knowledge Graph Service

5.6.1 Purpose

Represent the building as a semantic graph.

5.6.2 Node Types

- Building
- Floor
- Room
- Wall
- Door
- Window
- Beam
- Column
- Slab
- Stair
- Material

5.6.3 Relationships

- CONNECTED_TO
- INSIDE
- SUPPORTS
- CONTAINS
- ADJACENT_TO
- INTERSECTS
- HAS_OPENING
- ABOVE
- BELOW

5.6.4 Database

Neo4j

5.7 Quantity Engine

5.7.1 Purpose

Calculate engineering quantities automatically.

5.7.2 Responsibilities

- Concrete quantities
- Masonry quantities
- Excavation quantities
- Reinforcement estimation
- Finishing quantities
- Wall area
- Floor area
- Ceiling area
- Painting area
- Tile quantities

5.7.3 Outputs

- BOQ
- Quantity Tables
- Excel Export
- PDF Report
- JSON Output

5.8 Cost Engine

5.8.1 Purpose

Estimate construction costs.

5.8.2 Responsibilities

- Material Cost
- Labor Cost
- Equipment Cost
- Indirect Cost
- Profit Calculation
- Tax Calculation
- Regional Price Database

5.9 Reporting Service

5.9.1 Outputs

- PDF Reports
- Excel Reports
- CSV Reports
- JSON Reports
- Project Summary
- Quantity Tables
- Cost Tables
- Engineering Notes

Chapter 6

Project Architecture

6.1 Architecture Style

The system shall follow:

- Microservices Architecture
- Clean Architecture
- Domain Driven Design
- Event Driven Architecture
- API First Design
- AI First Design

6.2 Root Project Structure

```
bitonx/


 backend/

 frontend/

 ai/

 datasets/

 infrastructure/

 deployment/

 docs/

 scripts/

 tests/

 monitoring/
```

```
 shared/
```

6.3 Backend Structure

```
backend/


 app/
 api/
 auth/
 users/
 projects/
 upload/
 storage/
 reports/
 billing/
 notifications/
 settings/
 middleware/
 database/
 cache/
 queue/
 services/
 repositories/
 schemas/
 models/
 dependencies/
 utils/
```

```
 security/

 tests/
```

6.4 AI Structure

```
ai/


 preprocessing/

 enhancement/

 ocr/

 detection/

 segmentation/

 geometry/

 graph/

 quantity/

 cost/

 reasoning/

 llm/

 training/

 inference/

 evaluation/

 datasets/

 augmentation/

 models/

 checkpoints/

 experiments/

 configs/

 utils/
```

6.5 Frontend Structure

```
frontend/


 app/

 pages/

 components/

 layouts/

 hooks/

 services/

 api/

 store/

 utils/

 assets/

 styles/

 types/

 constants/

 tests/
```

Chapter 7

Dataset Design

7.1 Dataset Directory

```
datasets/


 raw/

 processed/

 annotations/

 metadata/

 reports/

 train/

 validation/

 test/

 masks/

 polygons/

 embeddings/

 statistics/
```

7.2 Supported Annotation Types

- Bounding Boxes
- Segmentation Masks
- Polygon Coordinates
- Keypoints

- OCR Labels
- Room Labels
- Scale Labels
- Material Labels
- Structural Labels

7.3 Object Classes

ID	Class
1	Wall
2	Door
3	Window
4	Column
5	Beam
6	Slab
7	Stair
8	Room
9	Corridor
10	Balcony
11	Elevator
12	Dimension
13	Text
14	Symbol
15	Furniture
16	Plumbing
17	Electrical
18	Roof
19	Foundation
20	Opening

Chapter 8

Training Pipeline

8.1 Training Workflow

```
Raw Dataset

↓

Validation

↓

Cleaning

↓

Augmentation

↓

Annotation Validation

↓

Train Split

↓

Model Training

↓

Validation

↓

Evaluation

↓

Checkpoint Saving
```

```
↓

Model Registry
```

8.2 Image Augmentation

- Rotation
- Flip
- Brightness
- Contrast
- Gaussian Noise
- Blur
- Scale
- Random Crop
- Perspective Transform
- Affine Transform

8.3 Evaluation Metrics

- Accuracy
- Precision
- Recall
- F1 Score
- IoU
- mAP
- Dice Score
- Confusion Matrix

8.4 Model Registry

Each trained model shall contain:

- Model Name
- Version
- Framework
- Dataset Version
- Training Date

- Hyperparameters
- Metrics
- Checkpoint Path
- Deployment Status

Chapter 9

Training Data Sources

9.1 Primary Datasets

The initial version of BitonX shall use the following public datasets for training, validation and benchmarking.

1. CubiCasa5K
2. ResPlan

9.2 Dataset Download Manager

The project shall automatically download datasets using KaggleHub.
Python Package:

- kagglehub

9.3 CubiCasa5K

Dataset Name:
CubiCasa5K
Provider:
Kaggle
Identifier:
qmarva/cubicasa5k
Purpose:

- Floor Plan Understanding
- Room Detection
- Wall Detection
- Door Detection
- Window Detection
- Symbol Detection
- Segmentation

9.4 ResPlan

Dataset Name:
ResPlan
Provider:
Kaggle
Identifier:
resplan/resplan
Purpose:

- Residential Floor Plans
- Layout Understanding
- Floor Classification
- Structural Learning
- Generalization

Chapter 10

Dataset Management

10.1 Dataset Directory

```
datasets/

raw/

cubicasa5k/

resplan/

processed/

cache/

annotations/

train/

validation/

test/

metadata/

statistics/

exports/
```

10.2 Dataset Loader

The Dataset Loader shall support:

- Automatic Download
- Dataset Versioning
- Local Cache
- Integrity Validation

- Automatic Extraction
- Missing File Detection
- Dataset Statistics

10.3 Dataset Validation

Every downloaded dataset shall be validated before training.

Validation Rules

- Missing Images
- Missing Labels
- Invalid Coordinates
- Corrupted Files
- Empty Annotations
- Duplicate Samples

Chapter 11

Dataset Preprocessing

The preprocessing pipeline shall perform:

1. Image Validation
2. Resolution Standardization
3. Annotation Validation
4. Polygon Repair
5. Coordinate Normalization
6. Duplicate Removal
7. Image Enhancement
8. Data Augmentation
9. Metadata Generation
10. Dataset Splitting

11.1 Image Resolution

Preferred Resolution
1024x1024
Supported Resolutions
512x512
768x768
1024x1024
1536x1536

11.2 Supported Annotation Types

- Bounding Boxes
- Segmentation Masks
- Polygon Vertices
- Room Labels
- Object Labels
- OCR Labels

- Structural Labels
- Material Labels

Chapter 12

Engineering Understanding Engine

The Engineering Understanding Engine is the core component of BitonX.

Unlike conventional object detectors, this module shall build an engineering representation of the uploaded drawing.

Responsibilities

- Understand structural layout
- Connect walls
- Connect openings
- Detect room topology
- Detect adjacency
- Detect circulation
- Validate geometry
- Estimate missing geometry
- Detect inconsistencies
- Generate engineering graph

12.1 Expected Output

The module shall generate

- Engineering Graph
- Room Graph
- Structural Graph
- Geometry Graph
- Semantic Building Model

12.2 Output Format

JSON
Graph
GeoJSON
CSV
PostgreSQL
Neo4j

Chapter 13

Engineering Computation Engine

13.1 Purpose

The Engineering Computation Engine is responsible for converting the semantic engineering model into measurable engineering quantities.

Input:

- Engineering Graph
- Geometry Graph
- Object Dimensions
- OCR Results
- Scale Information

Output:

- Quantity Tables
- BOQ
- Material Estimation
- Cost Estimation
- Engineering Report

13.2 Geometry Engine

Responsibilities

- Compute Length
- Compute Area
- Compute Volume
- Compute Perimeter
- Compute Height
- Compute Width
- Compute Thickness
- Compute Elevation

13.3 Wall Quantity Engine

Responsibilities

- Detect Wall Length
- Detect Wall Height
- Detect Wall Thickness
- Compute Wall Surface
- Compute Wall Volume
- Detect Openings
- Remove Door Area
- Remove Window Area
- Generate Wall BOQ

13.4 Floor Quantity Engine

Responsibilities

- Detect Floor Boundary
- Compute Gross Area
- Compute Net Area
- Compute Tile Quantity
- Compute Flooring Materials
- Compute Finishing Area

13.5 Ceiling Engine

Responsibilities

- Ceiling Area
- Ceiling Perimeter
- Gypsum Quantity
- Paint Quantity

13.6 Painting Engine

Responsibilities

- Wall Painting
- Ceiling Painting
- Exterior Painting
- Surface Preparation
- Paint Quantity

13.7 Concrete Engine

Responsibilities

- Foundation Volume
- Column Volume
- Beam Volume
- Slab Volume
- Stair Volume
- Total Concrete

13.8 Reinforcement Engine

Responsibilities

- Detect Reinforcement Information
- Estimate Reinforcement
- Compute Steel Weight
- Generate Reinforcement Table

13.9 Material Estimation Engine

Responsibilities

- Concrete
- Cement
- Sand
- Gravel
- Steel
- Brick
- Blocks
- Tiles
- Paint
- Plaster
- Wood
- Aluminum
- Glass

13.10 Cost Estimation Engine

Inputs

- Quantity Tables
- Material Prices
- Labor Prices
- Equipment Prices
- Regional Database

Outputs

- Material Cost
- Labor Cost
- Equipment Cost
- Total Cost
- Cost Breakdown

Chapter 14

Engineering Rules Engine

Purpose

Execute engineering rules independently from AI predictions.
Responsibilities

- Validate Dimensions
- Validate Geometry
- Detect Impossible Structures
- Detect Missing Walls
- Detect Invalid Rooms
- Detect Open Geometry
- Validate Connectivity
- Detect Engineering Errors
- Detect Inconsistent Measurements

Chapter 15

Engineering Reporting Engine

Output Formats

- PDF
- Excel
- CSV
- JSON

Reports

- Quantity Report
- BOQ
- Material Report
- Cost Report
- AI Analysis Report
- Geometry Report
- Validation Report

Chapter 16

AI Orchestration Pipeline

16.1 Objective

The AI Orchestration Pipeline coordinates every Artificial Intelligence module inside BitonX.
Each module receives structured outputs from the previous module.
No module shall operate independently.
Every prediction shall be validated before moving to the next stage.

16.2 Global Pipeline

```
User Upload

↓

File Validation

↓

Storage Service

↓

Vision Service

↓

Image Enhancement

↓

Drawing Classification

↓

OCR Service

↓

Scale Detection
```

```
↓

Object Detection

↓

Instance Segmentation

↓

Semantic Segmentation

↓

Geometry Reconstruction

↓

Relationship Extraction

↓

Engineering Graph Construction

↓

Engineering Reasoning

↓

Quantity Engine

↓

Material Estimation

↓

Cost Estimation

↓

BOQ Generation

↓

Engineering Validation

↓

Report Generation

↓
```

```
Frontend Dashboard
```

16.3 Stage 1 : File Validation

Responsibilities

- Validate MIME Type
- Validate Extension
- Validate File Size
- Scan Corrupted Files
- Detect Empty Documents
- Generate File Hash
- Store Metadata

16.4 Stage 2 : Image Enhancement

Responsibilities

- Noise Removal
- Image Sharpening
- Contrast Enhancement
- Histogram Equalization
- Deskew
- Border Detection
- Perspective Correction
- Image Normalization

Libraries

- OpenCV
- Pillow
- NumPy
- SciPy

16.5 Stage 3 : Drawing Classification

Purpose

Automatically determine the drawing type.
Possible Classes

- Floor Plan
- Structural Plan
- Electrical Plan
- Plumbing Plan
- HVAC Plan
- Roof Plan
- Elevation
- Section
- Site Plan

16.6 Stage 4 : OCR Pipeline

Responsibilities

- Detect Text
- Detect Dimensions
- Detect Symbols
- Detect Room Names
- Detect Notes
- Detect Scale
- Detect Grid Labels
- Detect Drawing Title

Expected Output
JSON

16.7 Stage 5 : Object Detection

Target Objects

- Wall
- Door
- Window
- Column
- Beam

- Slab
- Stair
- Furniture
- Symbol
- Equipment

Expected Output
Bounding Boxes
Confidence Scores
Object Classes

16.8 Stage 6 : Segmentation

Responsibilities

- Pixel Accurate Masks
- Wall Masks
- Room Masks
- Door Masks
- Window Masks
- Column Masks
- Beam Masks
- Stair Masks

16.9 Stage 7 : Geometry Reconstruction

Responsibilities

- Detect Lines
- Detect Corners
- Detect Intersections
- Detect Closed Rooms
- Detect Wall Centerlines
- Detect Room Boundaries
- Generate Polygon Geometry

Output
Engineering Geometry

16.10 Stage 8 : Engineering Graph

Responsibilities

- Connect Rooms
- Connect Walls
- Connect Openings
- Detect Adjacency
- Generate Graph Nodes
- Generate Graph Edges

Database
Neo4j

16.11 Stage 9 : Engineering Reasoning

Responsibilities

- Validate Building Logic
- Detect Missing Objects
- Infer Missing Walls
- Infer Missing Doors
- Validate Scale
- Validate Geometry
- Detect Impossible Structures
- Resolve Conflicts

16.12 Stage 10 : Quantity Computation

Responsibilities

- Area Computation
- Length Computation
- Volume Computation
- Surface Computation
- Material Quantities
- BOQ Generation
- Cost Inputs

16.13 Stage 11 : Engineering Validation

Responsibilities

- Validate Dimensions
- Validate Areas
- Validate Volumes
- Validate Quantities
- Validate Materials
- Validate Cost

16.14 Stage 12 : Report Generation

Generate

- Engineering Report
- BOQ
- Excel Report
- PDF Report
- CSV Export
- JSON Export

Chapter 17

Engineering Knowledge Base

17.1 Objective

The Engineering Knowledge Base stores engineering rules, construction concepts, material properties, unit definitions, relationships between building elements and quantity computation rules.

The Knowledge Base shall operate independently from AI models.

17.2 Knowledge Domains

- Architectural Engineering
- Structural Engineering
- Quantity Surveying
- Building Materials
- Construction Methods
- Engineering Standards
- Unit Conversion
- Cost Estimation
- Material Properties

17.3 Building Elements

The system shall maintain knowledge for:

- Building
- Floor
- Room
- Corridor
- Stair
- Elevator
- Wall
- Door

- Window
- Beam
- Column
- Slab
- Roof
- Foundation
- Footing
- Opening
- Ceiling
- Floor Finish
- Plumbing
- Electrical
- HVAC

Chapter 18

Database Architecture

18.1 Primary Database

Database
PostgreSQL
Purpose

- Users
- Projects
- Uploaded Files
- AI Predictions
- Quantities
- Cost Tables
- Reports
- Logs

18.2 Graph Database

Database
Neo4j
Purpose

- Building Graph
- Room Graph
- Connectivity
- Engineering Relationships
- Structural Dependencies

18.3 Object Storage

Storage
MinIO
Purpose

- Images
- PDFs
- AI Models
- Checkpoints
- Reports
- Dataset Cache

Chapter 19

Training Dataset Sources

The first version of BitonX shall use public datasets before collecting proprietary engineering data.
Primary Datasets

- CubiCasa5K
- ResPlan

Dataset Download Method
Python Package
kagglehub
The development environment shall support automatic dataset download and local caching.

19.1 Dataset Lifecycle

```
Kaggle

↓

Download

↓

Validation

↓

Extraction

↓

Cleaning

↓

Preprocessing

↓

Augmentation
```

```
↓

Annotation Validation

↓

Train

↓

Validation

↓

Testing

↓

Model Registry
```

Chapter 20

Core Database Tables

The PostgreSQL database shall include at least the following tables.

  - users
  - roles
  - permissions
  - projects
  - uploaded$_{f}iles$

- project$_{v}ersions$
- ai$_{j}obs$
- ai$_{p}redictions$
- detected$_{o}bjects$
- detected$_{r}ooms$
- detected$_{w}alls$
- detected$_{d}oors$
- detected$_{w}indows$
- geometry
- quantities
- boq
- material$_{c}osts$
- reports
- notifications
- audit$_{l}ogs$

Chapter 21

Engineering Graph

Node Types

- Building
- Floor
- Room
- Wall
- Door
- Window
- Beam
- Column
- Slab
- Material
- Opening

Relationships

- CONNECTED$_TO$
- CONTAINS
- INSIDE
- ABOVE
- BELOW
- SUPPORTS
- INTERSECTS
- HAS$_OPENING$
- ADJACENT$_TO$
- BUILT$_FROM$

Chapter 22

AI Model Registry

Each trained model shall contain:

- Model Name
- Version
- Framework
- Dataset Version
- Training Date
- Hyperparameters
- Metrics
- Checkpoint Path
- Deployment Status
- Author

Chapter 23

REST API Specification

23.1 API Principles

The backend shall expose a RESTful API.

API Version
/api/v1
Communication Format
JSON
Authentication
JWT Bearer Token
Transport
HTTPS
Encoding
UTF-8

23.2 Authentication Endpoints

Endpoint	Method	Description
/api/v1/auth/register	POST	Register User
/api/v1/auth/login	POST	Login
/api/v1/auth/logout	POST	Logout
/api/v1/auth/refresh	POST	Refresh Token
/api/v1/auth/profile	GET	Current User
/api/v1/auth/change-password	PUT	Change Password

23.3 Project Endpoints

Endpoint	Method	Description
/api/v1/projects	GET	List Projects
/api/v1/projects	POST	Create Project
/api/v1/projects/id	GET	Get Project
/api/v1/projects/id	PUT	Update Project
/api/v1/projects/id	DELETE	Delete Project

23.4 Upload Endpoints

Endpoint	Method	Description
/api/v1/upload	POST	Upload Drawing
/api/v1/upload/status/id	GET	Upload Status
/api/v1/upload/delete/id	DELETE	Delete File

23.5 AI Endpoints

Endpoint	Method	Description
/api/v1/vision/preprocess	POST	Image Processing
/api/v1/ocr/run	POST	OCR Execution
/api/v1/detection/run	POST	Object Detection
/api/v1/segmentation/run	POST	Segmentation
/api/v1/geometry/run	POST	Geometry Reconstruction
/api/v1/reasoning/run	POST	Engineering Reasoning
/api/v1/quantity/run	POST	Quantity Computation
/api/v1/cost/run	POST	Cost Estimation
/api/v1/report/run	POST	Report Generation

23.6 Reports

Endpoint	Method	Description
/api/v1/reports	GET	List Reports
/api/v1/reports/id	GET	Download Report
/api/v1/reports/id	DELETE	Delete Report

Chapter 24

API Error Codes

Code	Meaning
200	Success
201	Resource Created
400	Invalid Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
413	File Too Large
422	Validation Error
429	Too Many Requests
500	Internal Server Error
503	Service Unavailable

Chapter 25

Security Requirements

Authentication

- JWT Authentication
- OAuth2
- HTTPS Only
- Password Hashing
- Refresh Tokens

Authorization

- Role Based Access Control (RBAC)
- Permission Based Access Control

Protection

- SQL Injection Protection
- XSS Protection
- CSRF Protection
- Rate Limiting
- Audit Logging
- Secure File Upload

Chapter 26

Monitoring

Monitoring Stack

- Prometheus
- Grafana
- OpenTelemetry
- Loki
- Redis Insight

Metrics

- API Response Time
- CPU Usage
- GPU Usage
- Memory Usage
- Queue Length
- AI Inference Time
- OCR Accuracy
- Detection Accuracy
- Quantity Computation Time
- Cost Estimation Time

Chapter 27

AI Training Specification

27.1 Objective

The objective of the AI Training Framework is to train multiple specialized AI models instead of a single monolithic model.

Each model shall solve one engineering task.

27.2 Training Strategy

Training Methodology

- Transfer Learning
- Fine Tuning
- Mixed Precision Training
- Curriculum Learning
- Self Supervised Learning
- Semi Supervised Learning
- Knowledge Distillation
- Ensemble Learning

27.3 Drawing Classification Model

Purpose

Automatically classify engineering drawings.

Input

Drawing Image

Output

Drawing Type

Possible Classes

- Floor Plan
- Structural Plan
- Electrical Plan
- Plumbing Plan

- HVAC Plan
- Roof Plan
- Elevation
- Section

Candidate Models

- EfficientNet
- ConvNeXt
- Vision Transformer

27.4 Object Detection Model

Purpose

Detect engineering objects.

Training Dataset

- CubiCasa5K
- ResPlan
- Future Proprietary Dataset

Objects

- Walls
- Doors
- Windows
- Columns
- Beams
- Rooms
- Stairs
- Symbols

Candidate Models

- YOLO
- RT-DETR
- Faster RCNN

27.5 Segmentation Model

Purpose

Generate pixel accurate masks.

Candidate Models

- SAM
- Mask RCNN
- Mask2Former

27.6 OCR Model

Responsibilities

- Text Detection
- Text Recognition
- Dimension Extraction
- Room Labels
- Drawing Notes

Libraries

- PaddleOCR
- EasyOCR
- Tesseract

27.7 Geometry Reconstruction Model

Purpose
Transform detections into engineering geometry.
Input
Bounding Boxes
Segmentation Masks
OCR
Output
Polygon Geometry
Engineering Graph

27.8 Engineering Reasoning Model

Purpose
Infer engineering relationships.
Responsibilities

- Missing Walls
- Missing Doors
- Missing Windows
- Invalid Geometry
- Room Connectivity
- Structural Validation

Chapter 28

Training Infrastructure

Framework

PyTorch

Training Hardware

- NVIDIA GPU
- CUDA
- cuDNN
- Multi GPU

Experiment Tracking

- MLflow
- TensorBoard
- Weights and Biases

Chapter 29

Data Augmentation

Operations

- Rotation
- Horizontal Flip
- Vertical Flip
- Brightness
- Contrast
- Blur
- Gaussian Noise
- Perspective
- Affine Transform
- Scaling
- Random Crop
- Color Jitter

Chapter 30

Loss Functions

Classification
- Cross Entropy
- Detection
- Focal Loss
- Regression
- Smooth L1 Loss
- Segmentation
- Dice Loss
- Binary Cross Entropy
- Geometry
- Chamfer Distance
- Reasoning
- Graph Neural Network Loss

Chapter 31

Evaluation

Metrics

- Accuracy
- Precision
- Recall
- F1 Score
- mAP
- IoU
- Dice Score
- Confusion Matrix
- Mean Absolute Error
- Root Mean Square Error

Deployment Threshold

The deployment of a model shall require successful evaluation on validation and independent test datasets before being promoted to production.

Chapter 32

Implementation Roadmap

The implementation shall follow an incremental engineering strategy.
The AI Agent SHALL NOT skip any phase.
Every phase shall be completed before the next one begins.

32.1 Phase 1

Project Initialization
Tasks

- Create Repository
- Configure Git
- Configure Docker
- Configure Docker Compose
- Configure Python Environment
- Configure FastAPI
- Configure PostgreSQL
- Configure Neo4j
- Configure Redis
- Configure RabbitMQ
- Configure React
- Configure NextJS
- Configure CI/CD

Deliverable
Running Empty Platform

32.2 Phase 2

Dataset Pipeline
Tasks

- Download CubiCasa5K

- Download ResPlan
- Dataset Validation
- Dataset Cleaning
- Dataset Statistics
- Dataset Versioning
- Annotation Validation
- Dataset Splitting
- Data Augmentation

Deliverable
Training Ready Dataset

32.3 Phase 3

Vision Pipeline
Tasks

- PDF Processing
- Image Enhancement
- Perspective Correction
- Noise Removal
- Scale Detection
- Orientation Detection
- Metadata Extraction

Deliverable
Normalized Engineering Drawing

32.4 Phase 4

OCR Module
Tasks

- Text Detection
- Text Recognition
- Dimension Detection
- Room Labels
- Scale Detection
- Symbol Detection

Deliverable
Engineering Text JSON

32.5 Phase 5

Detection Module
Tasks

- Train Detection Model
- Validate Detection
- Detect Walls
- Detect Doors
- Detect Windows
- Detect Columns
- Detect Beams
- Detect Rooms
- Detect Stairs

Deliverable
Detected Objects

32.6 Phase 6

Segmentation Module
Tasks

- Train Segmentation
- Generate Masks
- Validate Masks
- Polygon Extraction

Deliverable
Engineering Masks

32.7 Phase 7

Geometry Engine
Tasks

- Polygon Reconstruction
- Line Detection
- Room Detection
- Wall Centerlines
- Corner Detection
- Topology
- Geometry Validation

Deliverable
Engineering Geometry

32.8 Phase 8

Engineering Graph
Tasks

- Room Graph
- Wall Graph
- Opening Graph
- Building Graph
- Neo4j Integration

Deliverable
Semantic Building Graph

32.9 Phase 9

Engineering Reasoning
Tasks

- Missing Object Detection
- Geometry Validation
- Relationship Validation
- Structural Validation
- Rule Engine

Deliverable
Validated Engineering Model

32.10 Phase 10

Quantity Engine
Tasks

- Length Calculation
- Area Calculation
- Volume Calculation
- Wall Quantities
- Concrete Quantities
- Tile Quantities
- Paint Quantities
- Steel Estimation

Deliverable
Complete Quantity Tables

32.11 Phase 11

Cost Engine
Tasks

- Material Prices
- Labor Prices
- Equipment Prices
- Cost Breakdown
- BOQ

Deliverable
Complete Cost Estimation

32.12 Phase 12

Reporting
Tasks

- PDF
- Excel
- CSV
- JSON
- Dashboard

Deliverable
Production Reports

32.13 Phase 13

Deployment
Tasks

- Docker Images
- Kubernetes
- Reverse Proxy
- Monitoring
- Logging
- Backup
- Production Testing

Deliverable
Production Platform

Chapter 33

AI Agent Instructions

The AI Agent SHALL

- Produce Production Code
- Produce Modular Code
- Produce Unit Tests
- Produce Integration Tests
- Produce Documentation
- Produce Docker Files
- Produce API Documentation
- Produce Logging
- Produce Monitoring
- Produce Type Hints
- Follow SOLID Principles
- Follow Clean Architecture
- Follow Domain Driven Design
- Avoid Hardcoded Values
- Write Reusable Components

The AI Agent SHALL NOT

- Skip Validation
- Duplicate Code
- Ignore Error Handling
- Ignore Logging
- Ignore Testing
- Ignore Documentation
- Ignore Security

Chapter 34

PostgreSQL Database Schema

34.1 Database Overview

Database Engine
- PostgreSQL 17+
- ORM
- SQLAlchemy 2.x
- Migration
- Alembic
- Primary Key
- UUID
- Timestamp Standard
- UTC
- Soft Delete
- Enabled

34.2 users

Purpose
- Store platform users.
- Columns

Column	Type	Description
id	UUID	Primary Key
first$_{n}ame$	VARCHAR(100)	User First Name
last$_{n}ame$	VARCHAR(100)	User Last Name
email	VARCHAR(255)	Unique Email
password$_{h}ash$	TEXT	Hashed Password
role$_{i}d$	UUID	User Role
is$_{a}ctive$	BOOLEAN	Active Status
last$_{l}ogin$	TIMESTAMP	Last Login
created$_{a}t$	TIMESTAMP	Creation Time
updated$_{a}t$	TIMESTAMP	Update Time
deleted$_{a}t$	TIMESTAMP	Soft Delete

34.3 roles

Columns

- id

- name
- description
- created$_at$

34.4 permissions

Columns

- id
- permission$_name$

- description

34.5 projects

Columns

- id
- owner$_id$

- project$_name$
- project$_type$
- status
- created$_at$
- updated$_at$

34.6 uploaded$_files$

Columns

- id
- project$_id$

- filename
- original$_name$
- mime$_type$
- file$_size$
- checksum
- storage$_path$
- uploaded$_at$

34.7 $\mathbf{ai}_{j}obs$

Columns

- id
- $\text{project}_{i}d$

- $\text{pipeline}_{s}tage$
- $\text{model}_{n}ame$
- $\text{model}_{v}ersion$
- status
- $\text{started}_{a}t$
- $\text{finished}_{a}t$
- $\text{execution}_{t}ime$

34.8 $\mathbf{detected}_{o}bjects$

Purpose
Store every detected engineering object.
Columns

- id
- $\text{project}_{i}d$

- $\text{page}_{n}umber$
- $\text{object}_{t}ype$
- confidence
- polygon
- $\text{bounding}_{b}ox$
- centroid_{x}
- centroid_{y}
- width
- height
- area
- perimeter
- rotation
- metadata
- $\text{created}_{a}t$

34.9 geometry

Columns

- id
- object$_id$

- geometry$_type$
- geometry$_json$
- length
- area
- volume
- thickness
- height

34.10 engineering$_graph$

Columns

- id
- object$_source$

- object$_target$
- relationship
- confidence
- metadata

34.11 material$_catalog$

Columns

- id
- material$_name$

- density
- unit
- category
- metadata

34.12 quantity$_{r}esults$

Columns

- id
- project$_{i}d$

- object$_{i}d$
- quantity$_{t}ype$
- value
- unit
- computation$_{m}ethod$
- created$_{a}t$

34.13 boq

Columns

- id
- project$_{i}d$

- item$_{c}ode$
- description
- quantity
- unit
- unit$_{p}rice$
- total$_{p}rice$

34.14 reports

Columns

- id
- project$_{i}d$

- report$_{t}ype$
- report$_{p}ath$
- generated$_{b}y$
- generated$_{a}t$

34.15 audit$_{l}ogs$

Columns

- id
- user$_{i}d$

- action
- resource
- ip$_{a}ddress$
- created$_{a}t$

34.16 Indexes

The following indexes shall be created.

- users(email)
- projects(owner$_{i}d$)

- uploaded$_{f}iles(project_{i}d)$
- ai$_{j}obs(project_{i}d)$
- detected$_{o}bjects(project_{i}d)$
- detected$_{o}bjects(object_{t}ype)$
- quantity$_{r}esults(project_{i}d)$
- boq(project$_{i}d$)

34.17 Constraints

The database shall enforce

- Foreign Keys
- Unique Constraints
- Check Constraints
- Cascade Delete Where Applicable
- Transaction Support
- Row Level Locking

Chapter 35

Neo4j Engineering Graph

35.1 Purpose

Neo4j shall store the semantic representation of every engineering drawing.

Each uploaded project shall generate an Engineering Knowledge Graph.

The graph shall represent spatial, structural and semantic relationships between all engineering entities.

35.2 Node Types

The following node labels shall be supported.

Node	Description
Building	Complete Building
Floor	Building Floor
Room	Room Entity
Wall	Wall Element
Door	Door Element
Window	Window Element
Column	Structural Column
Beam	Structural Beam
Slab	Concrete Slab
Roof	Roof Entity
Foundation	Foundation Entity
Stair	Stair Entity
Corridor	Corridor Entity
Opening	Opening Entity
Dimension	Dimension Annotation
Text	OCR Text
Material	Construction Material
Quantity	Computed Quantity
Cost	Cost Item
Report	Generated Report

35.3 Relationship Types

Relationship	Meaning
CONTAINS	Parent Contains Child
CONNECTED_TO	Physical Connection
ADJACENT_TO	Neighbor Element

INTERSECTS	Geometry Intersection
INSIDE	Located Inside
HAS_OPENING	Wall Has Door or Window
SUPPORTED_BY	Structural Support
ABOVE	Vertical Relation
BELOW	Vertical Relation
PART_OF	Component Relation
NEXT_TO	Neighborhood Relation
USES	Material Usage
GENERATES	Report or Quantity Relation
REFERENCES	Metadata Reference

35.4 Building Hierarchy

```
Building

↓

Floor

↓

Room

↓

Wall

↓

Opening

↓

Door

↓

Window
```

35.5 Wall Relationships

Every wall node may have:

- Adjacent Rooms
- Connected Walls
- Openings
- Materials
- Dimensions

- Quantities

35.6 Room Relationships

Every room shall maintain relationships with

- Walls
- Doors
- Windows
- Floor
- Ceiling
- Adjacent Rooms
- Area
- Perimeter

35.7 Node Properties

Every node shall include

- uuid
- project$_id$

- object$_type$
- confidence
- geometry
- metadata
- created$_at$
- updated$_at$

35.8 Relationship Properties

Every relationship shall contain

- uuid
- relationship$_type$

- confidence
- distance
- metadata
- created$_at$

35.9 Graph Construction Pipeline

The Engineering Graph shall be generated using the following sequence.

```
Object Detection

↓

Segmentation

↓

OCR

↓

Geometry Reconstruction

↓

Topology Extraction

↓

Relationship Inference

↓

Neo4j Graph Generation

↓

Engineering Validation
```

35.10 Cypher Query Examples

Example Operations

- Find all rooms.
- Find adjacent rooms.
- Find all doors connected to a room.
- Find walls containing openings.
- Find unsupported structural elements.
- Find isolated rooms.
- Compute room connectivity.
- Detect disconnected walls.

35.11 Graph Validation Rules

The graph validator shall verify

- No isolated walls
- No floating doors
- Every room is enclosed
- Every opening belongs to a wall
- Every wall belongs to a floor
- No duplicated nodes
- No duplicated relationships
- Geometry consistency

35.12 Future Extensions

The graph architecture shall support

- BIM Integration
- IFC Import
- IFC Export
- Digital Twin Generation
- 3D Reconstruction
- Structural Simulation
- Facility Management

Chapter 36

Frontend Architecture

36.1 Technology Stack

Framework
- Next.js 15
- Language
- TypeScript
- UI Library
- React 19
- Styling
- TailwindCSS
- Component Library
- shadcn/ui
- Icons
- Lucide React
- State Management
- Zustand
- Data Fetching
- TanStack Query
- Forms
- React Hook Form
- Validation
- Zod
- Charts
- Recharts
- Drawing Viewer
- Konva.js
- PDF Viewer
- PDF.js
- Notifications
- Sonner
- Animation
- Framer Motion
- Theme
- Dark / Light

36.2 Application Structure

```
app/
```

```
(auth)/

dashboard/

projects/

viewer/

reports/

settings/

api/

components/

features/

hooks/

lib/

services/

store/

types/

utils/

styles/

assets/
```

36.3 Main Pages

Page	Purpose
Landing Page	Product Presentation
Login	Authentication
Register	User Registration
Dashboard	Main Workspace
Projects	Project Management
Project Details	Project Information
Drawing Viewer	Drawing Visualization
AI Analysis	AI Results
BOQ	Quantity Tables
Cost Estimation	Cost Breakdown
Reports	Generated Reports
Notifications	User Notifications
Profile	User Profile
Settings	Application Settings

Administration	Platform Administration

36.4 Dashboard Layout

```
-------------------------------------------------

Top Navigation

-------------------------------------------------

Sidebar

Project Explorer

AI Status

Notifications

User Menu

-------------------------------------------------

Main Workspace

Drawing Viewer

Results Panel

Properties Panel

BOQ Panel

Report Panel

-------------------------------------------------

Footer
```

36.5 Sidebar Components

- Dashboard
- Projects
- Upload
- AI Analysis
- Quantities
- Cost Estimation
- Reports

- Settings
- Administration

36.6 Dashboard Widgets

- Active Projects
- AI Jobs
- Running Tasks
- GPU Usage
- Storage Usage
- Reports
- Recent Uploads
- Notifications

36.7 Drawing Viewer

Features

- Zoom
- Pan
- Rotate
- Fullscreen
- Layer Visibility
- Object Highlighting
- AI Overlay
- Wall Overlay
- Door Overlay
- Window Overlay
- Measurement Tool
- Selection Tool
- Grid
- Mini Map

36.8 AI Analysis Panel

Display

- OCR Results
- Object Detection
- Segmentation
- Geometry
- Engineering Graph
- Confidence Scores
- Validation Messages
- Processing Time

36.9 BOQ Interface

Display

- Material
- Unit
- Quantity
- Unit Price
- Total Price
- Export Excel
- Export PDF
- Search
- Filters
- Sorting

36.10 Report Viewer

Features

- PDF Preview
- Download
- Print
- Share
- Version History

36.11 State Management

Global Stores

- Auth Store
- User Store
- Project Store
- Upload Store
- AI Store
- Drawing Store
- Geometry Store
- BOQ Store
- Report Store
- Notification Store
- Settings Store

36.12 API Services

Frontend Services

- Auth Service
- Project Service
- Upload Service
- OCR Service
- Detection Service
- Segmentation Service
- Geometry Service
- Quantity Service
- Cost Service
- Report Service
- User Service

36.13 Routing

Public Routes

- /
- /login
- /register

Protected Routes

- /dashboard
- /projects
- /viewer
- /reports
- /settings
- /admin

36.14 Role Based UI

Administrator

- User Management
- AI Monitoring
- System Configuration
- Logs

Engineer

- Upload Drawings
- AI Analysis
- Quantity Reports
- Cost Reports

Viewer

- Read Reports
- Download Files
- View Projects

36.15 Theme System

Supported Themes

- Light
- Dark
- System Theme

36.16 Responsive Design

Supported Devices

- Desktop
- Laptop
- Tablet
- Mobile

36.17 Performance Requirements

The frontend shall

- Lazy Load Components
- Lazy Load Images
- Cache API Responses
- Virtualize Large Tables
- Optimize Rendering
- Support Offline Cache
- Compress Assets
- Use Code Splitting

36.18 Accessibility

The frontend shall comply with

- WCAG 2.2
- Keyboard Navigation
- Screen Reader Support
- High Contrast Mode
- Focus Indicators

Chapter 37

System Architecture and Infrastructure

37.1 Architecture Style

The BitonX platform shall follow a Modular Service-Oriented Architecture.
Core Principles

- Clean Architecture
- Domain Driven Design
- SOLID Principles
- Repository Pattern
- Dependency Injection
- Event Driven Communication
- Stateless APIs
- Scalable Infrastructure

37.2 Repository Structure

```
bitonx/

backend/

frontend/

ai/

datasets/

training/

models/

docker/

kubernetes/

nginx/
```

```
scripts/

monitoring/

docs/

tests/

.github/

README.md

docker-compose.yml

.env.example
```

37.3 Backend Structure

```
backend/

app/

api/

v1/

core/

config/

database/

models/

schemas/

repositories/

services/

vision/

ocr/

detection/

segmentation/

geometry/
```

```
reasoning/

quantity/

cost/

report/

workers/

middleware/

dependencies/

security/

tests/

main.py
```

37.4 Frontend Structure

```
frontend/

app/

components/

features/

hooks/

services/

store/

types/

styles/

assets/

utils/

public/
```

37.5 AI Structure

```
ai/
```

```
datasets/

preprocessing/

augmentation/

training/

evaluation/

inference/

vision/

ocr/

detection/

segmentation/

geometry/

reasoning/

models/

checkpoints/

weights/

metrics/

experiments/
```

37.6 Infrastructure Components

The infrastructure shall include

- FastAPI
- PostgreSQL
- Neo4j
- Redis
- RabbitMQ
- MinIO
- Nginx
- Docker
- Kubernetes

- Prometheus
- Grafana
- Loki
- MLflow

37.7 Application Services

Core Services

- Authentication Service
- User Service
- Project Service
- Upload Service
- OCR Service
- Vision Service
- Detection Service
- Segmentation Service
- Geometry Service
- Graph Service
- Engineering Reasoning Service
- Quantity Service
- Cost Service
- Reporting Service
- Notification Service
- AI Training Service
- Model Registry Service

37.8 Message Queue

RabbitMQ shall coordinate asynchronous processing.

Queue Types

- upload.queue
- preprocessing.queue
- ocr.queue
- detection.queue
- segmentation.queue
- geometry.queue

- reasoning.queue
- quantity.queue
- report.queue
- notification.queue

37.9 Execution Flow

```
Upload

↓

RabbitMQ

↓

Vision Worker

↓

OCR Worker

↓

Detection Worker

↓

Segmentation Worker

↓

Geometry Worker

↓

Reasoning Worker

↓

Quantity Worker

↓

Cost Worker

↓

Report Worker

↓
```

```
Frontend
```

37.10 Docker Services

Docker Compose shall include

- backend
- frontend
- postgres
- neo4j
- redis
- rabbitmq
- minio
- nginx
- prometheus
- grafana
- loki

37.11 Kubernetes

Production deployment shall support

- Deployment
- StatefulSet
- ConfigMap
- Secret
- Ingress
- Persistent Volume
- Horizontal Pod Autoscaler
- Service

37.12 Development Environments

Supported Environments

- Development
- Testing
- Staging
- Production

37.13 Continuous Integration

The CI pipeline shall perform

- Linting
- Formatting
- Static Analysis
- Unit Tests
- Integration Tests
- Security Scan
- Docker Build
- Artifact Generation

37.14 Continuous Deployment

Deployment Pipeline

- Build
- Test
- Package
- Push Images
- Deploy
- Smoke Tests
- Health Check
- Rollback Support

37.15 Logging

Logging shall include

- API Logs
- AI Logs
- Worker Logs
- Database Logs
- Security Logs
- Audit Logs

37.16 Monitoring

Metrics

- CPU
- GPU
- RAM
- API Latency
- Queue Length
- Database Performance
- AI Inference Time
- OCR Accuracy
- Detection Accuracy
- Segmentation Accuracy

37.17 Coding Standards

Python

- PEP8
- Ruff
- Black
- mypy
- pytest

TypeScript

- ESLint
- Prettier
- Strict Mode
- Type Safety

Git

- GitFlow
- Conventional Commits
- Pull Requests
- Code Review

37.18 Project Quality Requirements

Every component shall

- Be Modular
- Be Fully Typed
- Be Unit Tested
- Be Documented
- Be Reusable
- Be Secure
- Be Observable
- Be Scalable
- Be Production Ready

Chapter 38

Engineering Formula Engine

38.1 Objective

The Engineering Formula Engine (EFE) shall be responsible for all engineering quantity calculations performed by the BitonX platform.

Artificial Intelligence SHALL detect and reconstruct engineering objects, while the Formula Engine SHALL compute engineering quantities using deterministic mathematical rules.

The Formula Engine SHALL be independent from AI models.

38.2 Design Principles

The Formula Engine SHALL satisfy the following principles.

- Deterministic Computation
- Rule-Based Architecture
- Modular Formula Library
- Country Specific Rules
- Extensible Design
- Explainable Results
- Unit Validation
- Engineering Verification
- Full Traceability
- Version Controlled Formula Sets

38.3 Formula Categories

The engine shall support the following categories.

- Geometry
- Concrete
- Reinforcement Steel
- Masonry
- Excavation

- Backfilling
- Flooring
- Ceiling
- Painting
- Plaster
- Waterproofing
- Roofing
- Structural Elements
- Finishing
- Plumbing
- Electrical
- HVAC
- Cost Estimation

38.4 Geometry Engine

Supported Operations

- Distance
- Length
- Width
- Height
- Radius
- Diameter
- Perimeter
- Area
- Volume
- Polygon Area
- Polygon Perimeter
- Bounding Box
- Centroid
- Angle
- Slope

38.5 Concrete Formula Library

Supported Calculations

- Footings
- Strip Foundations
- Isolated Foundations
- Raft Foundations
- Columns
- Beams
- Slabs
- Stairs
- Walls
- Retaining Walls

38.6 Reinforcement Library

Supported Calculations

- Longitudinal Bars
- Stirrups
- Mesh
- Steel Weight
- Steel Density
- Bar Schedule
- Reinforcement Ratio

38.7 Masonry Library

Supported Calculations

- Block Quantity
- Brick Quantity
- Mortar Volume
- Wall Area
- Opening Deduction

38.8 Finishing Library

Supported Calculations

- Paint Quantity
- Tile Quantity
- Floor Finish
- Ceiling Finish
- Plaster Area
- Waterproofing Area

38.9 Unit Conversion Engine

Supported Units

- mm
- cm
- dm
- m
- km
- in
- ft
- yd
- m^2
- ft^2
- m^3
- ft^3
- kg
- ton
- L
- mL

38.10 Validation Engine

The engine shall verify

- Missing Dimensions
- Invalid Geometry
- Negative Values
- Unit Consistency
- Scale Consistency
- Formula Compatibility
- Duplicate Objects
- Engineering Constraints

38.11 Formula Execution Pipeline

```
Engineering Drawing

↓

AI Detection

↓

OCR

↓

Segmentation

↓

Geometry Reconstruction

↓

Engineering Graph

↓

Formula Selection

↓

Unit Conversion

↓

Quantity Calculation
```

```
↓

Engineering Validation

↓

BOQ Generation

↓

Cost Estimation

↓

Report Generation
```

38.12 Formula Registry

Each engineering formula shall contain

- Formula Identifier
- Formula Name
- Category
- Supported Objects
- Input Parameters
- Output Parameters
- Required Units
- Validation Rules
- Country Support
- Version
- Author
- Documentation

38.13 Country Rule Packs

The Formula Engine SHALL support independent rule packs.
Examples

- Morocco
- France
- Eurocode
- ACI

- ASTM
- British Standards
- GCC Standards

Each rule pack shall define

- Formula Overrides
- Material Standards
- Unit Standards
- Cost Rules
- Engineering Constraints

38.14 Explainable Engineering

Every generated quantity shall include

- Source Object
- Geometry Used
- Formula Applied
- Intermediate Values
- Final Result
- Unit
- Validation Status
- Confidence Level
- Execution Timestamp

38.15 Future Extensions

The Formula Engine architecture shall support

- BIM Integration
- IFC Native Computation
- Digital Twin Quantities
- Real-Time Cost Updates
- AI Assisted Formula Recommendation
- Custom Formula Editor
- Plugin Marketplace
- Cloud Formula Repository

Appendix A

Technology Stack and Version Matrix

A.1 Purpose

This appendix defines the official technology stack and version constraints for the BitonX platform.

All generated source code SHALL comply with the versions defined in this document unless explicitly updated in future releases.

A.2 Programming Languages

Technology	Version	Notes
Python	3.12.x	Backend and AI
TypeScript	5.x	Frontend
SQL	PostgreSQL 17	Database
Cypher	Latest Stable	Neo4j Queries
YAML	Latest Stable	Configuration Files
Markdown	Latest Stable	Documentation

A.3 Backend

Technology	Version	Purpose
FastAPI	Latest Stable	REST API
Uvicorn	Latest Stable	ASGI Server
SQLAlchemy	2.x	ORM
Alembic	Latest Stable	Database Migration
Pydantic	2.x	Validation
JWT	Latest Stable	Authentication
Redis	Latest Stable	Cache
RabbitMQ	Latest Stable	Queue
Celery	Latest Stable	Background Jobs
MinIO	Latest Stable	Object Storage

A.4 Artificial Intelligence

Technology	Version	Purpose
PyTorch	Latest Stable	Deep Learning
TorchVision	Latest Stable	Vision Models
OpenCV	Latest Stable	Computer Vision

NumPy	Latest Stable	Numerical Computing
Pandas	Latest Stable	Data Processing
Matplotlib	Latest Stable	Visualization
Scikit-learn	Latest Stable	Machine Learning
Albumentations	Latest Stable	Data Augmentation
ONNX Runtime	Latest Stable	Model Inference
OpenVINO	Latest Stable	CPU Optimization
PaddleOCR	Latest Stable	OCR
EasyOCR	Latest Stable	OCR Backup
Tesseract OCR	Latest Stable	OCR Fallback
MLflow	Latest Stable	Experiment Tracking
TensorBoard	Latest Stable	Training Visualization

A.5 Frontend

Technology	Version	Purpose
Next.js	15.x	Frontend Framework
React	19.x	UI Library
TypeScript	5.x	Programming Language
TailwindCSS	Latest Stable	Styling
shadcn/ui	Latest Stable	UI Components
TanStack Query	Latest Stable	Data Fetching
Zustand	Latest Stable	State Management
React Hook Form	Latest Stable	Forms
Zod	Latest Stable	Validation
Framer Motion	Latest Stable	Animations
Recharts	Latest Stable	Charts
Konva.js	Latest Stable	Drawing Viewer
PDF.js	Latest Stable	PDF Rendering

A.6 Databases

Technology	Version	Purpose
PostgreSQL	17.x	Relational Database
Neo4j	5.x	Graph Database
Redis	Latest Stable	Cache
MinIO	Latest Stable	Object Storage

A.7 DevOps

Technology	Version	Purpose
Docker	Latest Stable	Containerization
Docker Compose	Latest Stable	Local Deployment
Kubernetes	Latest Stable	Orchestration
Nginx	Latest Stable	Reverse Proxy
GitHub Actions	Latest Stable	CI/CD
Prometheus	Latest Stable	Monitoring
Grafana	Latest Stable	Dashboards
Loki	Latest Stable	Logging

A.8 Development Tools

Technology	Version	Purpose
Git	Latest Stable	Version Control
VS Code	Latest Stable	IDE
Cursor	Latest Stable	AI IDE
Antigravity	Latest Stable	AI Software Engineering
Overleaf	Latest Stable	Documentation
Postman	Latest Stable	API Testing
DBeaver	Latest Stable	Database Management

A.9 Official Datasets

The first production version of BitonX SHALL use the following datasets during model development.

- CubiCasa5K
- ResPlan
- Future Proprietary Engineering Dataset

A.10 Official Development Principles

The BitonX platform SHALL always follow:

- Clean Architecture
- Domain Driven Design
- SOLID Principles
- Modular Design
- Explainable Artificial Intelligence (XAI)
- Test Driven Development
- Secure by Design
- API First Development
- Production Ready Code
- Scalable Infrastructure

Part I

Engineering Governance and AI Development Contract

Appendix B

Project Scope (Version 1.0)

B.1 Mission

BitonX is an AI-powered engineering platform designed to automate construction quantity takeoff from architectural drawings using Computer Vision, Deep Learning and Engineering Intelligence.

B.2 Supported Features

Version 1.0 SHALL support

- 2D Architectural Drawings
- PDF Drawings
- PNG Drawings
- JPG Drawings
- Automatic Drawing Analysis
- OCR
- Object Detection
- Segmentation
- Geometry Reconstruction
- Quantity Takeoff
- BOQ Generation
- Cost Estimation
- Engineering Reports
- REST API
- Multi User Platform
- Cloud Deployment

B.3 Out of Scope

Version 1.0 SHALL NOT support

- BIM Authoring
- IFC Editing
- 3D Modeling
- Structural Analysis
- Finite Element Analysis
- MEP Analysis
- Real Time Collaboration
- Mobile Applications
- Offline Desktop Version

Appendix C

Engineering Rule Library

Every engineering quantity SHALL be calculated through deterministic engineering rules.

Artificial Intelligence SHALL NEVER directly estimate quantities without validation by the Formula Engine.

C.1 Rule Categories

- Geometry Rules
- Concrete Rules
- Steel Rules
- Masonry Rules
- Finishing Rules
- Excavation Rules
- Roofing Rules
- Plumbing Rules
- Electrical Rules
- HVAC Rules
- Cost Rules
- Unit Conversion Rules

C.2 Engineering Validation

The platform SHALL validate

- Drawing Scale
- Unit Consistency
- Geometry Consistency
- Area Validation
- Volume Validation
- Material Consistency

- Formula Compatibility
- Duplicate Objects
- Missing Objects
- Engineering Constraints

C.3 Explainable Engineering

Every engineering result SHALL include

- Source Drawing
- Source Objects
- Formula Used
- Intermediate Calculations
- Final Quantity
- Unit
- Confidence Score
- Validation Status
- Execution Time

Appendix D

Country Rule Packs

The Formula Engine SHALL support independent engineering packages.

- Morocco
- France
- Eurocode
- ACI
- ASTM
- GCC Standards
- Future Custom Plugins

Each package SHALL define

- Material Standards
- Formula Library
- Cost Rules
- Unit Standards
- Validation Rules
- Building Code Constraints

Appendix E

Product Roadmap

Version 1.0

- Architectural Drawings
- Quantity Takeoff
- Cost Estimation
- Engineering Reports

Version 2.0

- Structural Drawings
- BIM Import
- IFC Support
- AI Engineering Assistant
- Collaboration

Version 3.0

- Digital Twin
- 3D Reconstruction
- BIM Generation
- Construction Monitoring
- Predictive Analytics

Appendix F

AI Development Contract

Every AI Agent participating in the development of BitonX SHALL comply with the following mandatory requirements.

F.1 Architecture

The AI Agent SHALL

- Follow this specification strictly
- Preserve the system architecture
- Preserve modularity
- Preserve scalability
- Preserve security
- Preserve maintainability
- Preserve clean architecture
- Preserve SOLID principles

F.2 Implementation

The AI Agent SHALL

- Produce production-ready code
- Generate documentation
- Generate unit tests
- Generate integration tests
- Generate Docker configuration
- Generate API documentation
- Generate migration scripts
- Generate monitoring configuration
- Generate logging configuration

F.3 Engineering Constraints

The AI Agent SHALL NEVER

- Break backward compatibility
- Remove documented features
- Ignore engineering rules
- Ignore validation
- Hardcode engineering constants
- Skip testing
- Skip documentation
- Skip security verification
- Produce experimental production code

F.4 Final Objective

The final objective of BitonX is to become a fully autonomous AI Engineering Platform capable of understanding engineering drawings, reconstructing building geometry, applying engineering knowledge, performing deterministic quantity takeoff, generating Bills of Quantities, estimating project costs, and producing explainable engineering reports with production-grade reliability.